ReliefX : Smart Flood Relief Management System

Submitted By

Student Name	Student ID
Gulam Sakaria	242-15-218
MD. Ashraful Islam	242-15-487
Nafil Ardul Ridin	242-15-705
Mir Samiul Haque	242-15-350
Zidne Hasan	242-15-356

MINI LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course **CSE312: Database Management System Lab in the Computer Science and Engineering Department**

DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

August 17, 2026

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Mr. Shahadat Hossain**, **Assistant Professor**, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

____

Mr. Shahadat Hossain
Assistant Professor
Department of Computer Science and Engineering
Daffodil International University

Submitted by

Gulam Sakaria ID: 242-15-218 Dept. of CSE, DIU	
MD. Ashraful Islam ID: 242-15-487 Dept. of CSE, DIU	Nafil Ardul Ridin ID: 242-15-705 Dept. of CSE, DIU
Mir Samiul Haque ID: 242-15-350 Dept. of CSE, DIU	Zidne Hasan ID: 242-15-356 Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:

COs	CO Statements	POs	Learning Domains	Knowledge Profile	Complex Engineering Problem	Complex Engineering Activities
CO1	Implement and optimize relational databases using SQL queries, indexing techniques, and query optimization strategies.	PO3	C3, C5, A2, P1, P2	K4, K5, K6	EP1, EP4	
CO2	Apply and analyze database security techniques, distributed database architectures, and emerging database tools to address real-world database problems.	PO5	C3, C4, A2, P2, P3	K4, K6, K8	EP2	
CO3	Work effectively as an individual, team member, or leader in database project activities with proper planning and documentation.	PO9	A1, A2, A4, P3			
CO4	Demonstrate and present a database project effectively to communicate technical activities with the engineering community.	PO10	P2, A2, A4			EA1, EA2

The mapping justification of this table is provided in section **4.3.1**, **4.3.2** and **4.3.3**.

Table of Contents

Table of Contents

Chapter 1

Introduction

This chapter presents the background and motivation of the Relief Distribution Duplicate-Prevention System. It introduces the problem addressed by the project, its objectives, feasibility, identified gaps, and the expected outcomes of the developed system.

1.1 Introduction

Bangladesh is frequently affected by monsoon floods, creating an urgent need for food, medicine, shelter, and financial assistance. When multiple NGOs operate in the same area without centralized coordination, the same family may receive duplicate relief while others remain underserved. Directly storing NID numbers in paper or spreadsheet-based systems can also create privacy risks. To address these challenges, this project develops ReliefX: Smart Flood Relief Management System, a full-stack web application backed by a relational database. The system protects beneficiary identities using salted cryptographic hashing and prevents a family from receiving the same relief category within seven days through database-level triggers and a stored procedure. It also manages NGOs, relief items, stock, beneficiaries, distribution points, and distribution records, providing a more organized and transparent approach to flood-relief management.

1.2 Motivation

Traditional relief management becomes difficult when multiple NGOs operate simultaneously in the same disaster-affected area. Manual beneficiary verification and duplicate checking are time-consuming and may result in inefficient distribution of relief resources, where some families receive assistance multiple times while others remain underserved. The main motivation of ReliefX is to enforce important relief-management rules at the database level rather than relying only on the application layer. This helps maintain data integrity even when data is managed through different application interfaces or database operations. The project also provides an opportunity to apply important DBMS concepts in a realistic multi-entity environment, including normalization, primary and foreign key constraints, triggers, stored procedures, transactions, views, joins, aggregate functions, and subqueries. By combining these concepts, ReliefX aims to provide a more efficient, transparent, secure, and fair approach to emergency flood-relief management.

1.3 Objectives

The main objective of ReliefX: Smart Flood Relief Management System is to develop a secure, reliable, and efficient database-driven platform for managing flood-relief activities. The project aims to design a normalized relational database representing the administrative hierarchy of Bangladesh, including divisions, districts, upazilas, and affected areas, along with families, NGOs, relief items, stock, distribution points, and distribution records. It also focuses on protecting sensitive beneficiary information by storing identity numbers as salted cryptographic hashes instead of raw NID values. Another major objective is to enforce a database-level rule that prevents a family from receiving the same category of relief within seven days using stored procedures and triggers. The system also aims to prevent the same person from being registered as a family head in one record and a family member in another. In addition, ReliefX manages NGO relief inventory and updates available stock after successful distributions. The project further aims to provide a role-based web interface for administrators and NGO operators, maintain an audit trail of blocked duplicate attempts, and generate analytical reports through an area-wise fairness view. These objectives collectively aim to improve the efficiency, integrity, transparency, and fairness of emergency flood-relief management.

1.4 Project Outcome

The completed project provides a functional web-based ReliefX: Smart Flood Relief Management System with separate interfaces for administrative and NGO-operator activities. The MySQL database contains 14 relational tables, 2 views, 3 triggers, and 1 stored procedure, supporting beneficiary registration, NGO management, relief inventory, distribution, duplicate detection, auditing, and reporting. The system protects beneficiary identities through hashed storage and prevents duplicate relief distribution using database-level rules. Blocked duplicate attempts are recorded in the duplicate_log table, while NGO stock is updated after successful distributions. The v_area_fairness view also helps analyze relief distribution in relation to the population of affected areas. Overall, ReliefX demonstrates the practical application of database engineering principles to improve data integrity, resource management, privacy, duplicate prevention, transparency, and fairness in flood-relief management.

Chapter 2

Proposed Architecture

This chapter presents the proposed architecture and design of ReliefX: Smart Flood Relief Management System. It describes the system requirements, overall methodology, database and application architecture, user interface design, and the overall project plan.

2.1 Requirement Analysis & Design Specification

2.1.1 Overview

ReliefX is designed as a web-based flood-relief management system that allows administrators and NGO operators to manage affected families, relief items, NGO stock, distribution activities, and related records. The system uses a centralized MySQL database to maintain data consistency and enforce important business rules such as duplicate relief prevention and stock validation.

The major functional and non-functional requirements of the system are described below.

Functional Requirements

The system should provide the following major functionalities:

- **Family Registration:** Register flood-affected families and store their identity information using salted cryptographic hashing.
- **Family Management:** View and update beneficiary information when required.
- **NGO Management:** Maintain NGO information and their associated activities.
- **Relief Item Management:** Maintain different categories of relief items such as food, medical, shelter, and cash.
- **Stock Management:** Maintain NGO-specific relief stock and update available quantities after successful distribution.
- **Relief Distribution:** Record relief distributed to registered families through designated distribution points.
- **Duplicate Prevention:** Prevent a family from receiving the same relief category within seven days.
- **Duplicate Logging:** Record blocked duplicate distribution attempts for auditing.
- **Role-Based Access:** Support separate Admin and NGO Operator activities.
- **Analytical Reporting:** Generate reports such as NGO-wise distribution and area-wise relief fairness.
- **Family Verification:** Prevent the same person from being registered as a family head and as a member of another family.

Non-Functional Requirements

The system should also satisfy the following requirements:

- **Security:** Sensitive beneficiary identity information should not be stored as raw NID values.
- **Data Integrity:** Primary keys, foreign keys, unique constraints, and validation rules should maintain consistent data.
- **Reliability:** Database-level rules should prevent invalid or duplicate distribution records.
- **Scalability:** The relational structure should support additional NGOs, families, areas, and relief items.
- **Usability:** The web interface should allow users to perform common operations easily.
- **Maintainability:** The system should use a structured database and modular application design for easier maintenance.

2.1.2 ERD Diagram

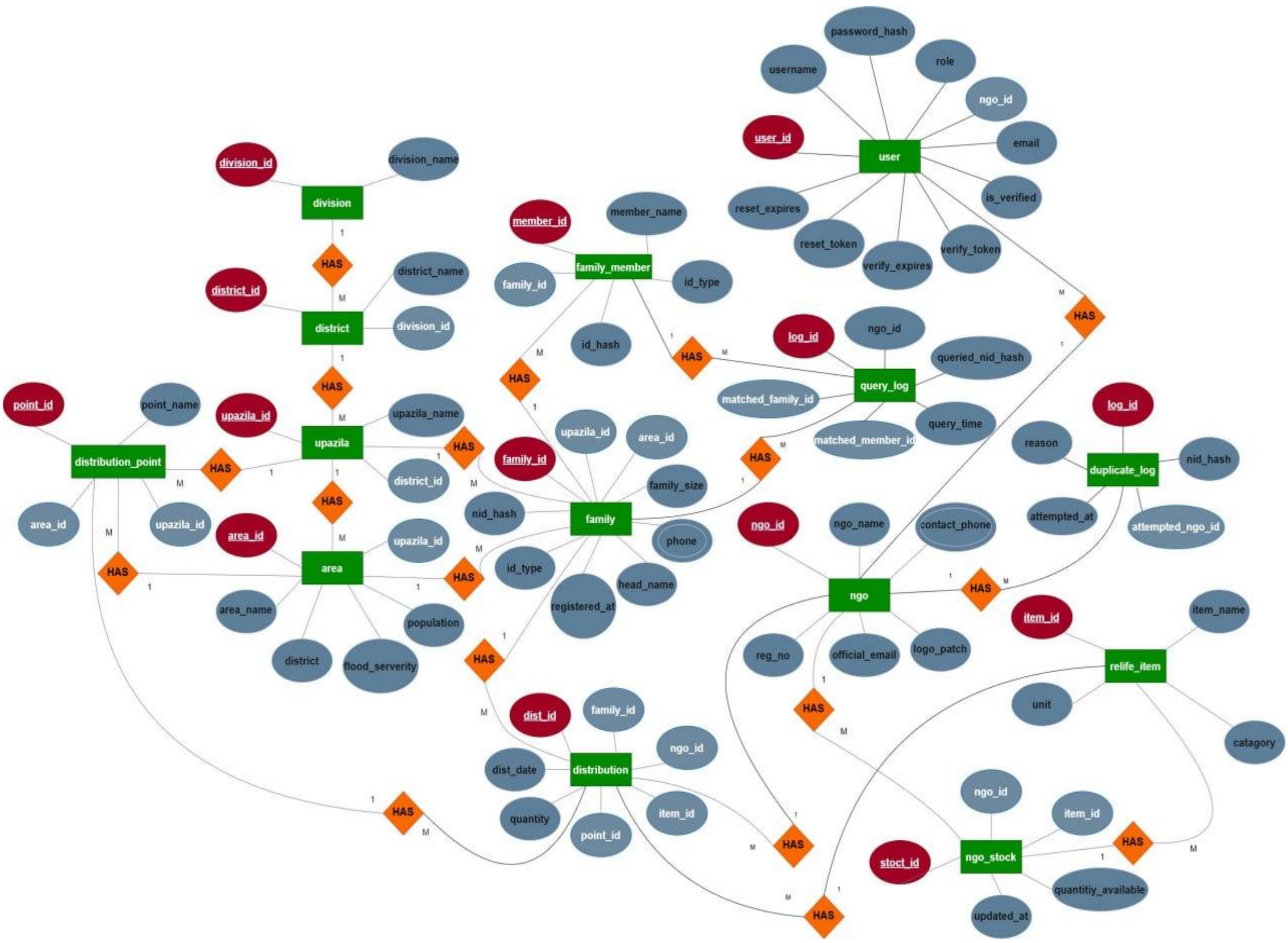


Figure 2.1: Entity-Relationship Diagram (ERD) of the ReliefX Smart Flood Relief Management System

2.1.3 UI Design

The user interface of ReliefX: Smart Flood Relief Management System is designed to provide simple, role-based, and organized access to the system's functionalities. The interface is divided according to user roles so that administrators can manage system-level operations, while NGO operators can perform activities related to relief distribution and monitoring.

Login Interface :

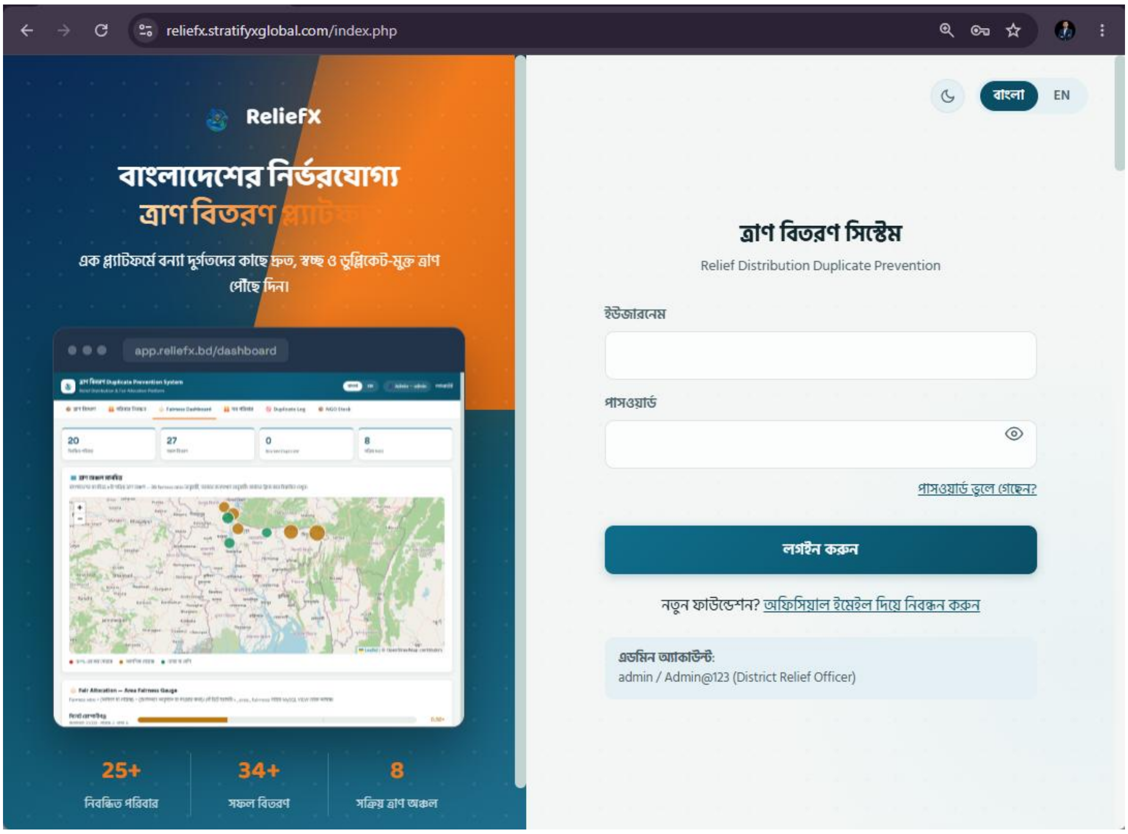


Figure 2.2 Login Interface

Admin Interface :

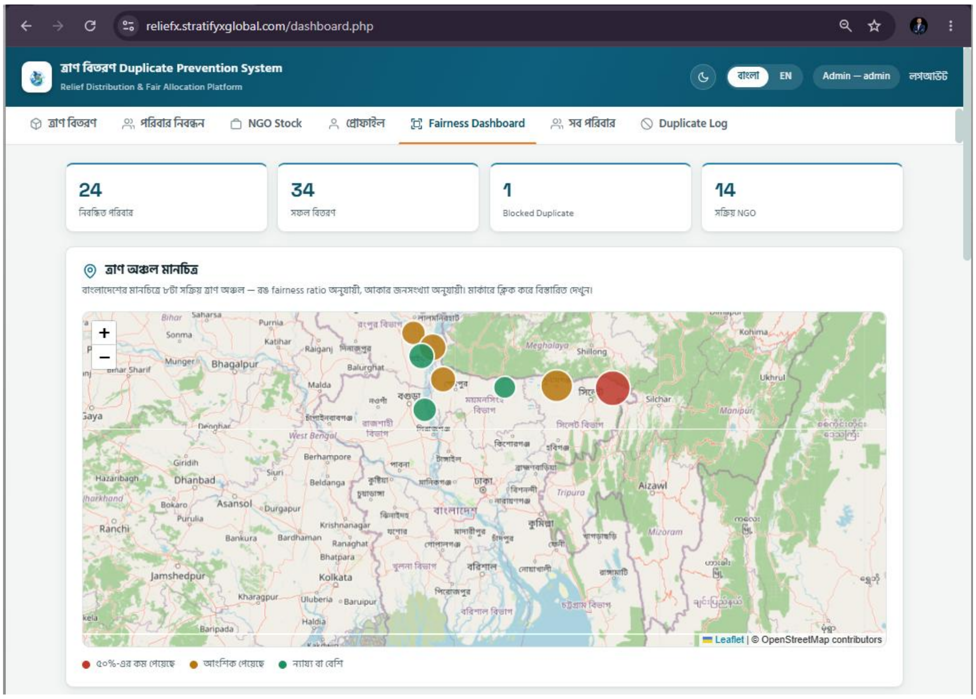


Figure 2.3: Admin Dashboard of ReliefX

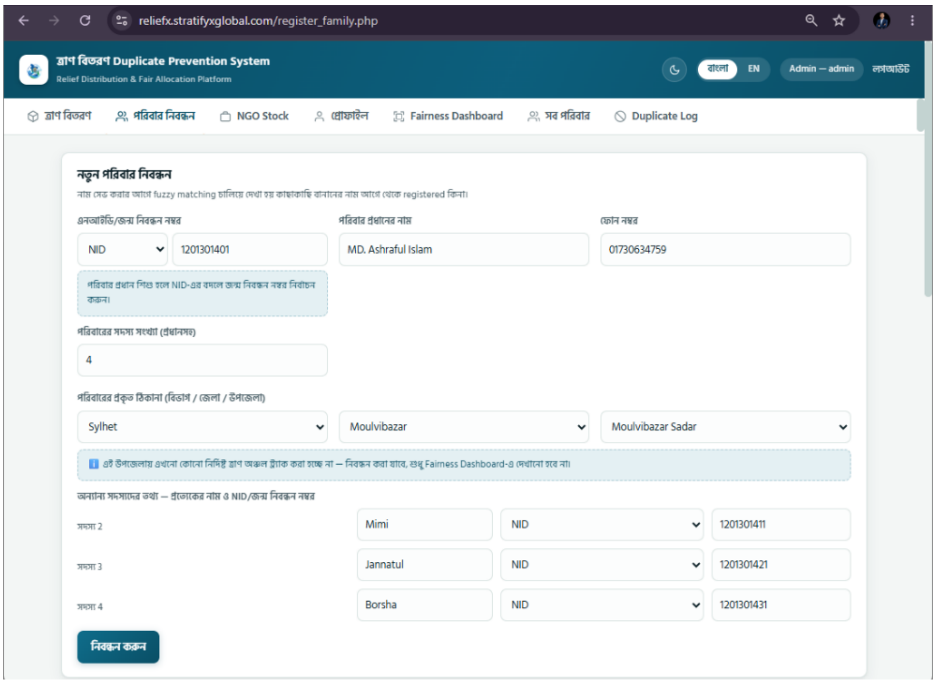


Figure 2.4: Family Registration Interface of ReliefX

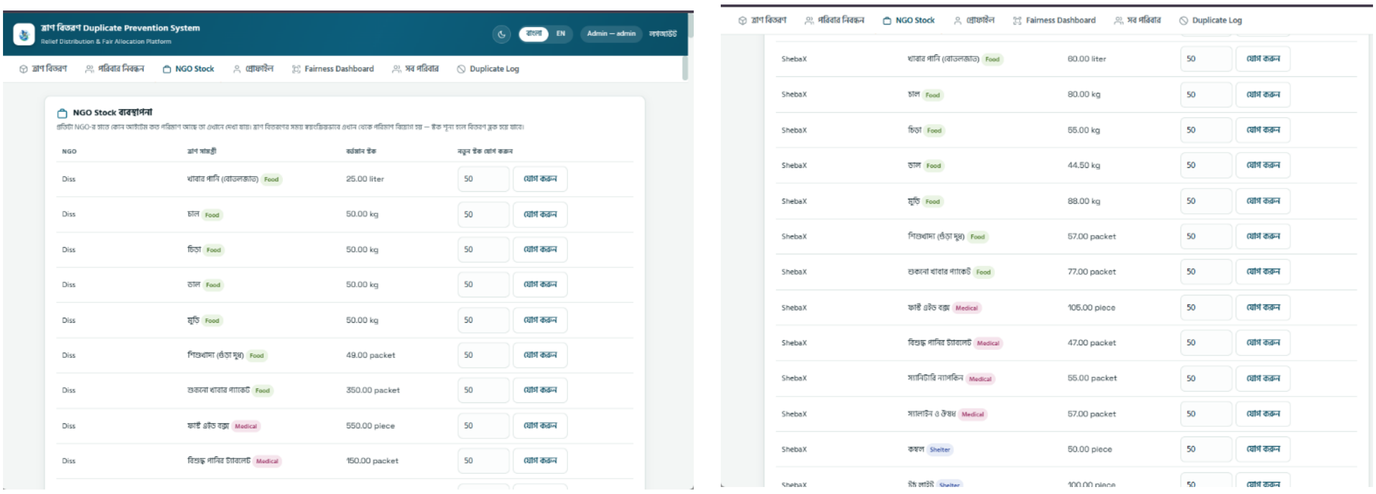


Figure 2.5: Relief Item and Stock Management Interface

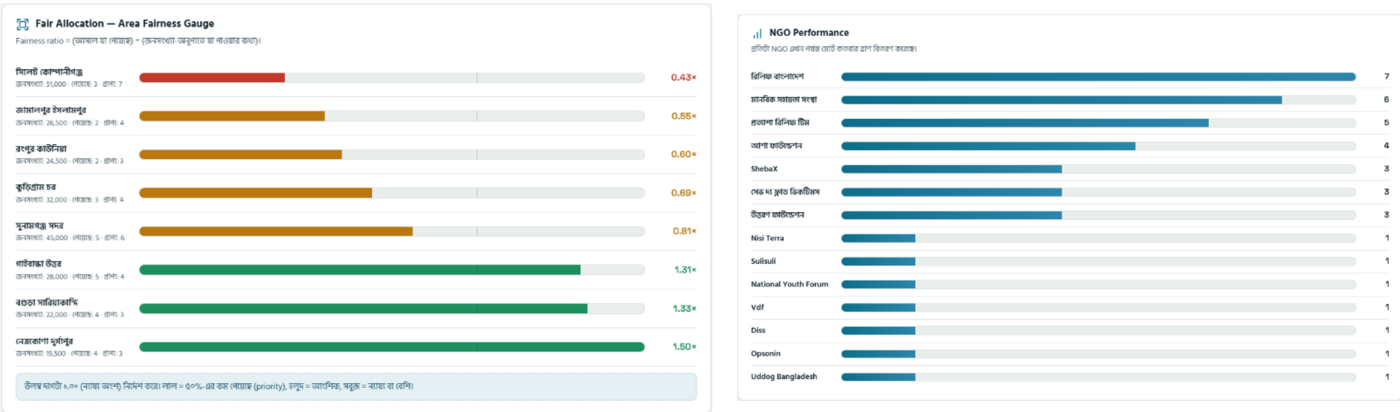


Figure 2.6: Area-wise Fairness Report Interface

NGO Operator Interface :

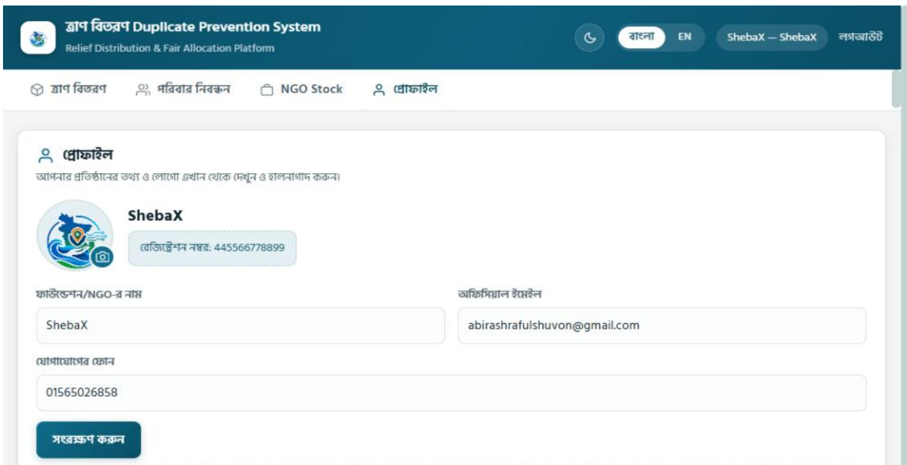


Figure 2.7: NGO Operator Profile

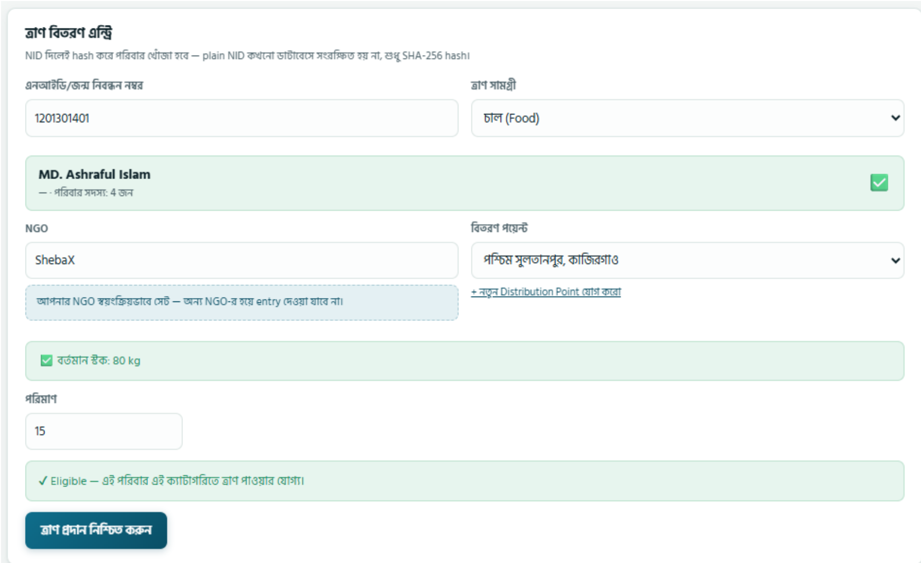


Figure 2.8: Relief Distribution Interface

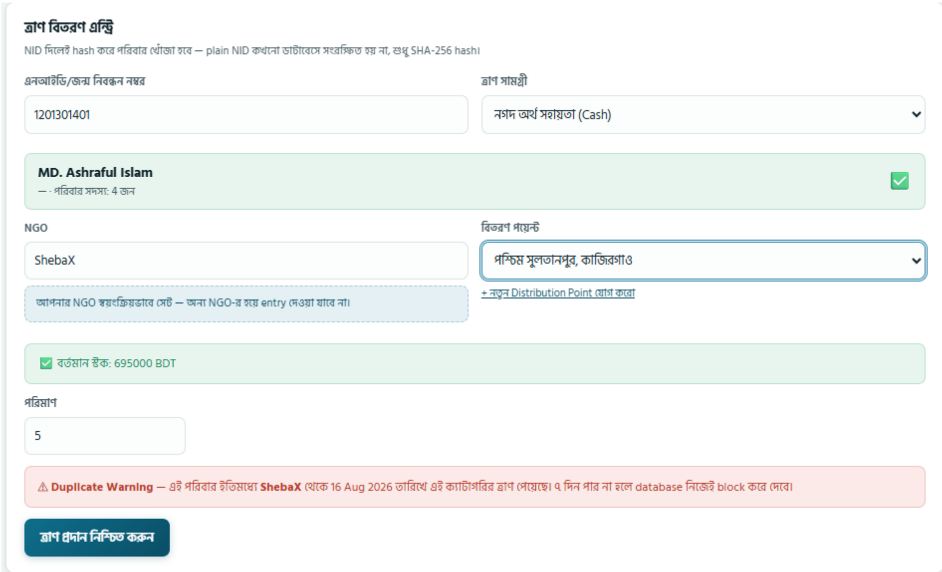


Figure 2.9: Relief Distribution Duplicate Warning

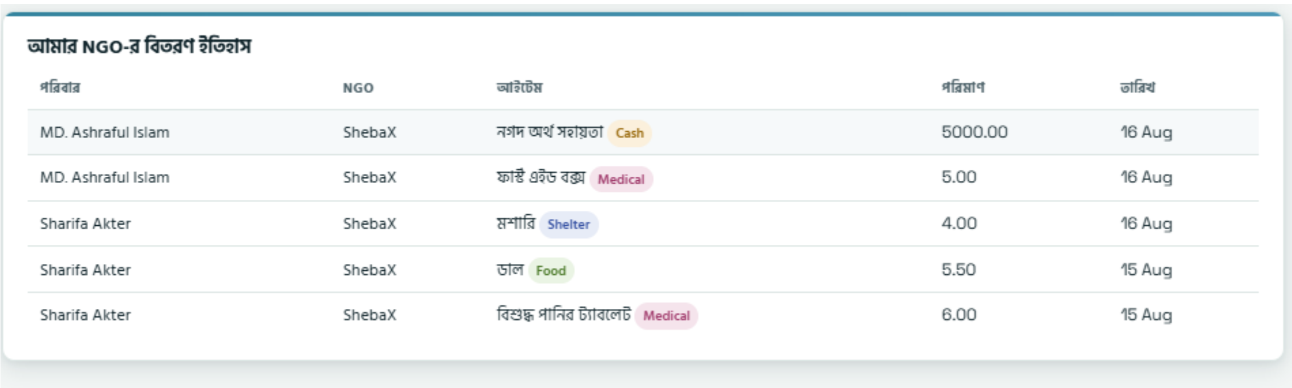


Figure 2.10: Distribution Records Interface

2.2 Overall Project Plan

The development of ReliefX: Smart Flood Relief Management System was completed through several systematic phases, covering requirement analysis, database development, backend and UI implementation, testing, and final reporting.

Phase 1: Requirement Analysis
The system requirements were identified based on key flood-relief management challenges, including duplicate distribution, beneficiary management, stock tracking, privacy, and fairness analysis.

Phase 2: Database Design
The relational database was designed by defining entities, relationships, keys, constraints, and normalization requirements.

Phase 3: Database Implementation
The MySQL database was implemented with tables, relationships, indexes, views, triggers, stored procedure, and sample data.

Phase 4: Backend & UI Development
The PHP backend and web interfaces were developed to support authentication, beneficiary management, stock management, relief distribution, and role-based operations.

Phase 5: Integration & Testing
The frontend, backend, and database were integrated and tested using different system scenarios, including successful distribution, insufficient stock, and duplicate distribution attempts.

Phase 6: Reporting & Finalization
Analytical queries and reports were prepared for NGO-wise distribution, uncovered families, critical areas, and area-wise fairness, followed by final system review and documentation.

Chapter 3

Implementation and Results

This chapter presents the implementation of ReliefX: Smart Flood Relief Management System and discusses the results obtained from the implemented database operations and system functionalities. The implementation section focuses on the SQL-based database development, while the results section presents the outputs and observations obtained from testing the system.

3.1 Implementation

```sql
DROP DATABASE IF EXISTS relief_db;
CREATE DATABASE relief_db CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
USE relief_db;


CREATE TABLE family (
    family_id INT AUTO_INCREMENT PRIMARY KEY,
    nid_hash CHAR(64) NOT NULL UNIQUE,
    id_type ENUM('NID','Birth Certificate') NOT NULL DEFAULT 'NID',
    head_name VARCHAR(100) NOT NULL,
    phone VARCHAR(15),
    family_size INT DEFAULT 1 CHECK (family_size BETWEEN 1 AND 20),
    area_id INT NULL,
    upazila_id INT NULL,
    registered_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (area_id) REFERENCES area(area_id) ON DELETE RESTRICT,
    FOREIGN KEY (upazila_id) REFERENCES upazila(upazila_id)
) ENGINE=InnoDB;
```

Figure 3.1: Database and Family Table Creation in MySQL

```sql
CREATE TABLE ngo_stock (
    stock_id INT AUTO_INCREMENT PRIMARY KEY,
    ngo_id INT NOT NULL,
    item_id INT NOT NULL,
    quantity_available DECIMAL(10,2) NOT NULL DEFAULT 0 CHECK
(quantity_available >= 0),
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    UNIQUE KEY uq_ngo_item (ngo_id, item_id),
    FOREIGN KEY (ngo_id) REFERENCES ngo(ngo_id),
    FOREIGN KEY (item_id) REFERENCES relief_item(item_id)
) ENGINE=InnoDB;

CREATE TABLE distribution (
    dist_id INT AUTO_INCREMENT PRIMARY KEY,
    family_id INT NOT NULL,
    ngo_id INT NOT NULL,
    item_id INT NOT NULL,
    point_id INT NOT NULL,
    quantity DECIMAL(8,2) NOT NULL CHECK (quantity > 0),
    dist_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (family_id) REFERENCES family(family_id),
    FOREIGN KEY (ngo_id)    REFERENCES ngo(ngo_id),
    FOREIGN KEY (item_id)   REFERENCES relief_item(item_id),
    FOREIGN KEY (point_id)  REFERENCES distribution_point(point_id)
) ENGINE=InnoDB;
```

Figure 3.2: NGO Stock and Relief Distribution Table Implementation

```sql
CREATE VIEW v_person_registry AS
SELECT family_id, nid_hash AS id_hash, head_name
AS person_name, 'Head' AS person_role, NULL AS
member_id
FROM family
UNION ALL
SELECT family_id, id_hash, member_name AS
person_name, 'Member' AS person_role, member_id
FROM family_member;
```

Figure 3.3: Person Registry View for Unified ID Verification

```sql
-- TRIGGER
CREATE TRIGGER trg_block_duplicate
BEFORE INSERT ON distribution
FOR EACH ROW
BEGIN
    DECLARE recent_count INT;

    -- JOIN
    SELECT COUNT(*) INTO recent_count
    FROM distribution d
    JOIN relief_item r1 ON d.item_id = r1.item_id
    JOIN relief_item r2 ON r2.item_id = NEW.item_id
    WHERE d.family_id = NEW.family_id
      AND r1.category = r2.category
      AND d.dist_date >= NOW() - INTERVAL 7 DAY;

    IF recent_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'DUPLICATE BLOCKED: This
family already received this category within 7 days';
    END IF;
END //

```

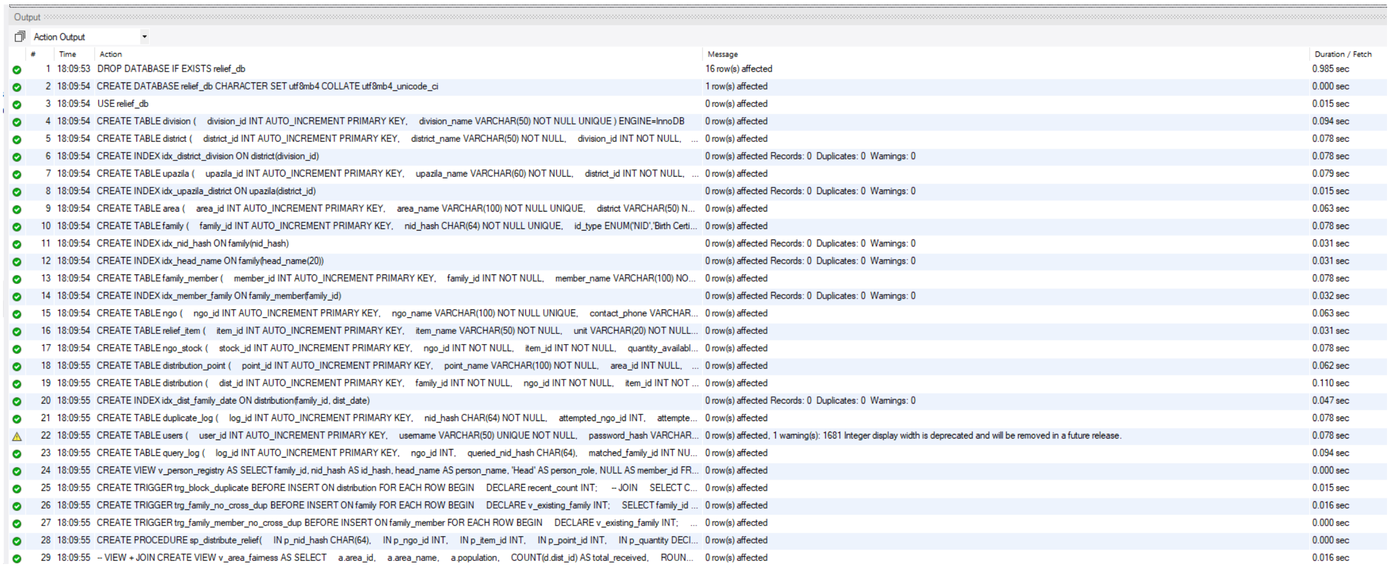
Figure 3.4: Trigger for Preventing Duplicate Relief Distribution

```sql
-- TRANSACTION
    START TRANSACTION;

    SELECT family_id INTO v_family_id
    FROM v_person_registry WHERE id_hash = p_nid_hash LIMIT 1;

    IF v_family_id IS NULL THEN
        SET p_status = 'ERROR: Family not registered';
        COMMIT;
        LEAVE proc_body;
    END IF;

    SELECT family_id INTO v_family_id FROM family WHERE family_id = v_family_id
FOR UPDATE;

    -- JOIN
    SELECT COUNT(*) INTO v_recent
    FROM distribution d
    JOIN relief_item r1 ON d.item_id = r1.item_id
    JOIN relief_item r2 ON r2.item_id = p_item_id
    WHERE d.family_id = v_family_id
      AND r1.category = r2.category
      AND d.dist_date >= NOW() - INTERVAL 7 DAY;

    IF v_recent > 0 THEN
        INSERT INTO duplicate_log (nid_hash, attempted_ngo_id, reason)
        VALUES (p_nid_hash, p_ngo_id, 'Same category within 7 days');
        SET p_status = 'BLOCKED: Duplicate attempt logged';
        COMMIT;
        LEAVE proc_body;
    END IF;

    SELECT quantity_available INTO v_stock
    FROM ngo_stock WHERE ngo_id = p_ngo_id AND item_id = p_item_id FOR UPDATE;

    IF v_stock IS NULL OR v_stock < p_quantity THEN
        SET p_status = 'ERROR: Insufficient stock for this NGO/item';
        ROLLBACK;
        LEAVE proc_body;
    END IF;

    UPDATE ngo_stock SET quantity_available = quantity_available - p_quantity
    WHERE ngo_id = p_ngo_id AND item_id = p_item_id;

    INSERT INTO distribution (family_id, ngo_id, item_id, point_id, quantity)
    VALUES (v_family_id, p_ngo_id, p_item_id, p_point_id, p_quantity);
    SET p_status = 'SUCCESS: Relief distributed';
    COMMIT;

```

Figure 3.5: Stored Procedure for Transaction-Based Relief Distribution

3.2 Output

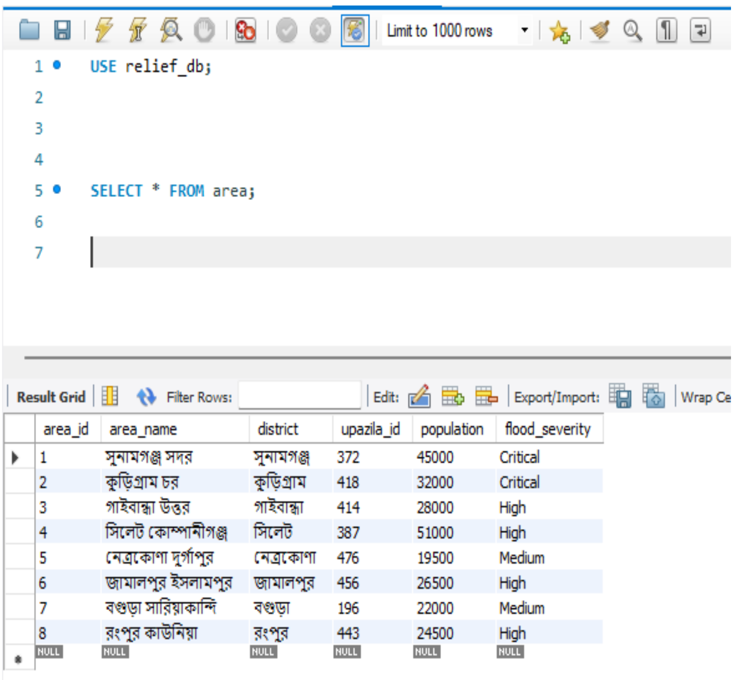
Result 1 : Database & Table Creation



	#	Time	Action	Message	Duration / Fetch
✓	1	18:09:53	DROP DATABASE IF EXISTS relief_db	16 row(s) affected	0.985 sec
✓	2	18:09:54	CREATE DATABASE relief_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci	1 row(s) affected	0.000 sec
✓	3	18:09:54	USE relief_db	0 row(s) affected	0.015 sec
✓	4	18:09:54	CREATE TABLE division (division_id INT AUTO_INCREMENT PRIMARY KEY, division_name VARCHAR(50) NOT NULL UNIQUE) ENGINE=InnoDB	0 row(s) affected	0.094 sec
✓	5	18:09:54	CREATE TABLE district (district_id INT AUTO_INCREMENT PRIMARY KEY, district_name VARCHAR(50) NOT NULL, division_id INT NOT NULL, ...	0 row(s) affected	0.078 sec
✓	6	18:09:54	CREATE INDEX idx_district_division ON district(division_id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.078 sec
✓	7	18:09:54	CREATE TABLE upazila (upazila_id INT AUTO_INCREMENT PRIMARY KEY, upazila_name VARCHAR(60) NOT NULL, district_id INT NOT NULL, ...	0 row(s) affected	0.079 sec
✓	8	18:09:54	CREATE INDEX idx_upazila_district ON upazila(district_id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.015 sec
✓	9	18:09:54	CREATE TABLE area (area_id INT AUTO_INCREMENT PRIMARY KEY, area_name VARCHAR(100) NOT NULL UNIQUE, district VARCHAR(50) N...	0 row(s) affected	0.063 sec
✓	10	18:09:54	CREATE TABLE family (family_id INT AUTO_INCREMENT PRIMARY KEY, nid_hash CHAR(64) NOT NULL UNIQUE, id_type ENUM('NID','Birth Certi...	0 row(s) affected	0.078 sec
✓	11	18:09:54	CREATE INDEX idx_nid_hash ON family(nid_hash)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec
✓	12	18:09:54	CREATE INDEX idx_head_name ON family(head_name(20))	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec
✓	13	18:09:54	CREATE TABLE family_member (member_id INT AUTO_INCREMENT PRIMARY KEY, family_id INT NOT NULL, member_name VARCHAR(100) NO...	0 row(s) affected	0.078 sec
✓	14	18:09:54	CREATE INDEX idx_member_family ON family_member(family_id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.032 sec
✓	15	18:09:54	CREATE TABLE ngo (ngo_id INT AUTO_INCREMENT PRIMARY KEY, ngo_name VARCHAR(100) NOT NULL UNIQUE, contact_phone VARCHAR...	0 row(s) affected	0.063 sec
✓	16	18:09:54	CREATE TABLE relief_item (item_id INT AUTO_INCREMENT PRIMARY KEY, item_name VARCHAR(50) NOT NULL, unit VARCHAR(20) NOT NULL...	0 row(s) affected	0.031 sec
✓	17	18:09:54	CREATE TABLE ngo_stock (stock_id INT AUTO_INCREMENT PRIMARY KEY, ngo_id INT NOT NULL, item_id INT NOT NULL, quantity_availabl...	0 row(s) affected	0.078 sec
✓	18	18:09:55	CREATE TABLE distribution_point (point_id INT AUTO_INCREMENT PRIMARY KEY, point_name VARCHAR(100) NOT NULL, area_id INT NULL, ...	0 row(s) affected	0.062 sec
✓	19	18:09:55	CREATE TABLE distribution (dist_id INT AUTO_INCREMENT PRIMARY KEY, family_id INT NOT NULL, ngo_id INT NOT NULL, item_id INT NOT ...	0 row(s) affected	0.110 sec
✓	20	18:09:55	CREATE INDEX idx_dist_family_date ON distribution(family_id, dist_date)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
✓	21	18:09:55	CREATE TABLE duplicate_log (log_id INT AUTO_INCREMENT PRIMARY KEY, nid_hash CHAR(64) NOT NULL, attempted_ngo_id INT, attempte...	0 row(s) affected	0.078 sec
⚠	22	18:09:55	CREATE TABLE users (user_id INT AUTO_INCREMENT PRIMARY KEY, username VARCHAR(50) UNIQUE NOT NULL, password_hash VARCHAR...	0 row(s) affected, 1 warning(s): 1681 Integer display width is deprecated and will be removed in a future release.	0.078 sec
✓	23	18:09:55	CREATE TABLE query_log (log_id INT AUTO_INCREMENT PRIMARY KEY, ngo_id INT, queried_nid_hash CHAR(64), matched_family_id INT NU...	0 row(s) affected	0.094 sec
✓	24	18:09:55	CREATE VIEW v_person_registry AS SELECT family_id, nid_hash AS id_hash, head_name AS person_name, 'Head' AS person_role, NULL AS member_id FR...	0 row(s) affected	0.000 sec
✓	25	18:09:55	CREATE TRIGGER trg_block_duplicate BEFORE INSERT ON distribution FOR EACH ROW BEGIN DECLARE recent_count INT; -- JOIN SELECT C...	0 row(s) affected	0.015 sec
✓	26	18:09:55	CREATE TRIGGER trg_family_no_cross_dup BEFORE INSERT ON family FOR EACH ROW BEGIN DECLARE v_existing_family INT; SELECT family_id ...	0 row(s) affected	0.016 sec
✓	27	18:09:55	CREATE TRIGGER trg_family_member_no_cross_dup BEFORE INSERT ON family_member FOR EACH ROW BEGIN DECLARE v_existing_family INT; ...	0 row(s) affected	0.000 sec
✓	28	18:09:55	CREATE PROCEDURE sp_distribute_relief(IN p_nid_hash CHAR(64), IN p_ngo_id INT, IN p_item_id INT, IN p_point_id INT, IN p_quantity DECI...	0 row(s) affected	0.000 sec
✓	29	18:09:55	-- VIEW + JOIN CREATE VIEW v_area_fairness AS SELECT a.area_id, a.area_name, a.population, COUNT(d.dist_id) AS total_received, ROUN...	0 row(s) affected	0.016 sec

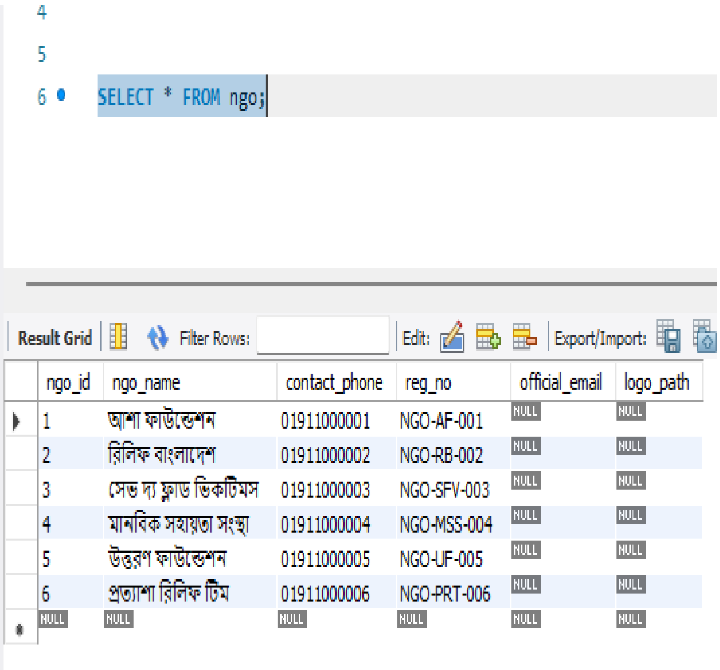
Figure 3.6: Successfully Created ReliefX Database and Tables

Result 2 : Data Insertion & Record Verification




```
USE relief_db;



SELECT * FROM area;

```

area_id	area_name	district	upazila_id	population	flood_severity
1	সুনামগঞ্জ সদর	সুনামগঞ্জ	372	45000	Critical
2	কুড়িগ্রাম চর	কুড়িগ্রাম	418	32000	Critical
3	গাইবান্ধা উত্তর	গাইবান্ধা	414	28000	High
4	সিলেট কোম্পানীগঞ্জ	সিলেট	387	51000	High
5	নেত্রকোণা দুর্গাপুর	নেত্রকোণা	476	19500	Medium
6	জামালপুর ইসলামপুর	জামালপুর	456	26500	High
7	বগুড়া সারিয়াকান্দি	বগুড়া	196	22000	Medium
8	রংপুর কাউনিয়া	রংপুর	443	24500	High
NULL	NULL	NULL	NULL	NULL	NULL




```
SELECT * FROM ngo;
```

ngo_id	ngo_name	contact_phone	reg_no	official_email	logo_path
1	আশা ফাউন্ডেশন	01911000001	NGO-AF-001	NULL	NULL
2	রিলিফ বাংলাদেশ	01911000002	NGO-RB-002	NULL	NULL
3	সেভ দ্য ফ্লাড ভিকটিমস	01911000003	NGO-SFV-003	NULL	NULL
4	মানবিক সহায়তা সংস্থা	01911000004	NGO-MSS-004	NULL	NULL
5	উত্তরণ ফাউন্ডেশন	01911000005	NGO-UF-005	NULL	NULL
6	প্রত্যাশা রিলিফ টিম	01911000006	NGO-PRT-006	NULL	NULL
NULL	NULL	NULL	NULL	NULL	NULL

```sql
SELECT * FROM relief_item;
```

item_id	item_name	unit	category
1	চাল	kg	Food
2	ডাল	kg	Food
3	শুকনো খাবার প্যাকেট	packet	Food
4	স্যালাইন ও ঔষধ	packet	Medical
5	কম্বল	piece	Shelter
6	তারপুলিন/ত্রিপল	piece	Shelter
7	বিশুদ্ধ পানির ট্যাবলেট	packet	Medical
8	নগদ অর্থ সহায়তা	BDT	Cash
9	চিড়া	kg	Food
10	মুড়ি	kg	Food
11	খাবার পানি (বোতলজ...	liter	Food
12	শিশুখাদ্য (গুঁড়া দুধ)	packet	Food
13	স্যানিটারি ন্যাপকিন	packet	Medical
14	ফার্স্ট এইড বক্স	piece	Medical
15	মশারি	piece	Shelter
16	মোমবাতি ও দিয়াশলাই	packet	Shelter
17	টর্চ লাইট	piece	Shelter
18	দড়ি	piece	Shelter
NULL	NULL	NULL	NULL

```sql
SELECT * FROM family;
```

family_id	nid_hash	id_type	head_name	phone	family_size	area_id	upazila_id	registered_at
1	84f85c8b447b5e6a0317b67e9d342d8c9172b8d...	NID	রহিম উদ্দিন	01711000001	5	1	NULL	2026-07-29 18:09:55
2	4c6123efa3f73ea0961ca4cdbaeae8723f9ce40b...	NID	করিম মিয়া	01711000002	4	2	NULL	2026-07-26 18:09:55
3	7fdf4c05e78f9b4e94161d1fbb4b15cd8e32c4e5...	NID	ফাতেমা বেগম	01711000003	6	3	NULL	2026-07-23 18:09:55
4	4db3518b8ff7fd93c49b0b18033a3b5ed6df98a...	NID	জোবেদা খাতুন	01711000004	3	4	NULL	2026-07-20 18:09:55
5	72c0e8be25db68ef13fadc765fd96e3497479ab...	NID	আব্দুল হালিম	01711000005	5	5	NULL	2026-07-17 18:09:55
6	312d1b40764e6956338f6140bf7ce2843bb791d...	NID	সালমা আক্তার	01711000006	4	6	NULL	2026-07-14 18:09:55
7	df7fdb8a4ed16c50978c18bcc29445580381918f...	NID	নুরুল ইসলাম	01711000007	7	7	NULL	2026-07-11 18:09:55
8	1410ba6aef371f4101ec7900e952f76d601a164...	NID	রোকেয়া বেগম	01711000008	2	8	NULL	2026-07-08 18:09:55
9	91779478dc4389a49b25ef27785a939decb309a...	NID	মোজাম্মেল হক	01711000009	4	1	NULL	2026-07-05 18:09:55
10	5e3bab0531e570319474337385c53fd9793664...	NID	তাহেরা খাতুন	01711000010	3	2	NULL	2026-07-02 18:09:55
11	af5b8cb6e01c52b004277159498364584acf47e...	NID	আনোয়ার হো...	01711000011	5	3	NULL	2026-06-29 18:09:55
12	9bcf8d510e29fb57abab1ac98b12c34be53ce20...	NID	মরিয়ম বিবি	01711000012	6	4	NULL	2026-06-26 18:09:55
13	b7cab269c4bd95126c30dc9536377682f1b23a8...	NID	শাহজাহান আলী	01711000013	4	5	NULL	2026-06-23 18:09:55
14	03cfb1e4527af727af4c2ab3f09952573bab28c2...	NID	নাসরিন আক্তার	01711000014	3	6	NULL	2026-07-30 18:09:55
15	9b2f2cf85eea1d8d4fcf038a47c0b1ccb49ef4a3...	NID	দেলোয়ার হো...	01711000015	5	7	NULL	2026-07-27 18:09:55
16	e4ccd53874084f0b87231295ee6aafadddf2f049...	NID	রাবেয়া খাতুন	01711000016	4	8	NULL	2026-07-24 18:09:55
17	1aee72ae99d1882cfe0fc2e00bdc804c095280d...	NID	ইউসুফ আলী	01711000017	6	1	NULL	2026-07-21 18:09:55
18	f7caadc90348ef89cfa90004b7fb5986fd9ccdff6...	NID	সুরাইয়া বেগম	01711000018	3	2	NULL	2026-07-18 18:09:55
19	a14519dad86ae760d12aa6c1a114357431256a...	NID	জাহাঙ্গীর আলম	01711000019	5	3	NULL	2026-07-15 18:09:55
20	cb54af5beb6eb3f8f40a062e45eca3bed4a6453...	NID	হালিমা খাতুন	01711000020	4	4	NULL	2026-07-12 18:09:55
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 3.7: Sample ReliefX Records After Data Insertion

Result 3 : Successful Relief Distribution

```sql
USE relief_db;
SELECT *
FROM ngo_stock
WHERE ngo_id = 1
AND item_id = 1;

SELECT *
FROM distribution
ORDER BY dist_id DESC;
```

dist_id	family_id	ngo_id	item_id	point_id	quantity	dist_date
28	15	4	4	9	6.00	2026-08-14 18:09:55
27	13	2	2	7	8.00	2026-07-16 18:09:55
26	11	6	7	4	10.00	2026-07-22 18:09:55
25	9	4	3	1	6.00	2026-07-28 18:09:55
24	7	2	6	9	8.00	2026-08-04 18:09:55
23	5	6	1	7	10.00	2026-08-10 18:09:55
22	3	4	8	4	6.00	2026-08-13 18:09:55
21	1	2	5	1	8.00	2026-08-14 18:09:55
20	20	2	4	5	5.00	2026-07-15 18:09:55
19	19	1	3	4	11.00	2026-07-18 18:09:55
18	18	6	2	3	9.00	2026-07-21 18:09:55
17	17	5	1	1	7.00	2026-07-24 18:09:55
16	16	4	8	10	5.00	2026-07-27 18:09:55
15	15	3	7	9	11.00	2026-07-30 18:09:55
14	14	2	6	8	9.00	2026-08-02 18:09:55
13	13	1	5	7	7.00	2026-06-26 18:09:55
12	12	6	4	5	5.00	2026-06-29 18:09:55
11	11	5	3	4	11.00	2026-07-02 18:09:55
10	10	4	2	3	9.00	2026-07-05 18:09:55
9	9	3	1	1	7.00	2026-07-08 18:09:55
8	8	2	8	10	5.00	2026-07-11 18:09:55
7	7	1	7	9	11.00	2026-07-14 18:09:55
6	6	6	6	8	9.00	2026-07-17 18:09:55
5	5	5	5	7	7.00	2026-07-20 18:09:55
4	4	4	4	5	5.00	2026-07-23 18:09:55
3	3	3	3	4	11.00	2026-07-26 18:09:55
2	2	2	2	3	9.00	2026-07-29 18:09:55
1	1	1	1	1	7.00	2026-08-01 18:09:55
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 3.8: Successful Relief Distribution and Stock Update

Result 4 : Duplicate Distribution Blocked

@status
BLOCKED: Duplicate attempt logged

log_id	nid_hash	attempted_ngo_id	attempted_at	reason
1	84f85c8b447b5e6a0317b67e9d342d8c9172b8d...	2	2026-08-16 18:25:41	Same category within 7 days
NULL	NULL	NULL	NULL	NULL

Figure 3.9: Duplicate Relief Distribution Prevention Result

Result 5 : Insufficient Stock Handling

@status
ERROR: Insufficient stock for this NGO/item

ngo_id	item_id	quantity_available
1	1	500.00

Figure 3.10: Insufficient Stock Validation Result

Result 6 : Area-Based Fairness Analysis

area_id	area_name	population	total_received	expected_share	fairness_ratio
4	সিলেট কোম্পানীগঞ্জ	51000	3	6	0.52
6	জামালপুর ইসলামপুর	26500	2	3	0.67
8	রংপুর কাউনিয়া	24500	2	3	0.72
2	কুড়িগ্রাম চর	32000	3	4	0.83
1	সুনামগঞ্জ সদর	45000	5	5	0.99
3	গাইবান্ধা উত্তর	28000	5	3	1.58
7	বগুড়া সারিয়াকান্দি	22000	4	2	1.61
5	নেত্রকোণা দুর্গাপুর	19500	4	2	1.82

Figure 3.11: Area-Based Relief Distribution Fairness Analysis

3.3 Result and Discussion

The implemented ReliefX database was successfully tested with sample data and different database operations. The results showed that relief distribution could be completed successfully with proper stock updates, while duplicate distributions and insufficient-stock requests were automatically blocked. The fairness analysis also provided area-wise distribution information to evaluate the balance of relief allocation. Overall, the results demonstrate that the implemented database operations work as intended and support reliable and controlled flood relief management.

Chapter 4

Engineering Standards and Mapping

This chapter discusses the engineering, social, ethical, and sustainability aspects of ReliefX: Smart Flood Relief Management System. It also presents the project management approach, team responsibilities, and estimated project cost.

4.1 Impact on Society, Environment and Sustainability

4.1.1 Impact on Life

ReliefX: Smart Flood Relief Management System can help improve the efficiency of emergency relief distribution during flood situations. By reducing duplicate distribution and maintaining organized beneficiary and stock records, the system can help ensure that limited relief resources reach more affected families. The system can therefore contribute to faster, more organized, and fairer relief management.

4.1.2 Impact on Society & Environment

The system supports better coordination among NGOs by maintaining centralized beneficiary, distribution, and inventory information. This can reduce unnecessary duplication of relief resources and improve resource utilization. As a digital system, it also reduces dependence on paper-based record keeping, which can help minimize paper consumption and physical documentation.

4.1.3 Ethical Aspects

Privacy and data security were considered important aspects of the project. Instead of storing raw NID numbers, the system uses salted cryptographic hashing for identity information. The system also uses role-based access so that administrative and NGO operations can be controlled according to user responsibilities. Furthermore, duplicate-prevention and audit mechanisms help promote transparency and accountability in relief distribution.

4.1.4 Sustainability Plan

The system is designed using commonly available technologies such as PHP, MySQL, HTML, CSS, and JavaScript, making it relatively easy to maintain and extend. The database structure can be expanded with additional NGOs, relief categories, affected areas, and reporting features when required. Future improvements may include cloud deployment, mobile accessibility, automated notifications, and more advanced data analytics.

4.2 Project Management and Team Work

The ReliefX project was completed through collaborative teamwork, with each team member taking responsibility for specific aspects of the system development. The major responsibilities were distributed as follows:

Gulam Sakaria (Team Leader) - Backend Development, Database Schema Design , Project planning & team coordination.
MD. Ashraful Islam - Requirement Analysis , Proposal, Database Implementation, QA & Documentation.
Nafil Ardul Ridin – Query for report, Role Management, ER Diagram & Normalization.
Mir Samiul Haque - Feature development, UI/UX Design, Frontend & DB Integration Check.
Zidne Hasan - Bug Fixing, Query Engineering ,System and SQL testing, Testing & Presentation.

Cost Analysis :
Since ReliefX was developed as an academic project, the team mainly used existing computing resources and freely available development tools. Therefore, the project required minimal direct financial investment.

Project Costing

Relief Distribution Duplicate Prevention System

SL	Item Description	Duration	Cost (BDT)
01	Shared Domain	1 Year	2,500
02	Shared Hosting	1 Year	1,500
03	Anthropic (Claude) Usage	1 Month	2,800
04	ChatGPT Usage	1 Month	2,600
05	Project Proposal, Documentation & Report	One-time	500
06	SMS Gateway API (NGO)	Monthly	5,000
	Total		**14,900**

Note: All amounts are stated in Bangladeshi Taka (BDT).

4.3 Complex Engineering Problem

4.3.1 Complex Problem Solving

This section presents how the ReliefX project addresses complex engineering problems through systematic analysis, database-driven solutions, and integrated system development. The complex problem attributes identified in the project are mapped to the requirements specified in the guideline using Table 4.2. The engineering activities performed to address these challenges are presented in Table 4.3, along with a brief justification of how each activity contributed to solving the identified problems.

Table 4.2: Mapping with complex engineering problem.

#	CEP Attribute	Tick (✓)	Justification
1	**EP1: Depth of knowledge required** – Cannot be resolved without in-depth engineering knowledge at the level of one or more of K3, K4, K5, K6 or K8 which allows a fundamentals-based, first principles analytical approach.	✓	The project required in depth engineering knowledge to design and implement subqueries, SQL coding, normalization up to Third Normal Form (3NF), multi-table joins, constraints, triggers, stored procedures, and transactions in order to solve the database-level problems. This level of analytical, fundamentals based engineering knowledge satisfies the EP1 criterion.
2	**EP2: Range of conflicting requirements** – Involve wide-ranging or conflicting technical, engineering and other issues.	✓	The project had to balance conflicting requirements: protecting beneficiary privacy by avoiding storage of raw NID values, against the need for fast, searchable identity lookup resolved through salted hashing and balancing system usability against strict data integrity, enforced through server-side triggers and validation rather than relying only on frontend checks. Resolving these conflicting requirements through practical engineering trade-offs satisfies the EP2 criterion.
4	**EP4: Familiarity of issues** – Involve infrequently encountered issues.	✓	Handling hashed NID verification, cross-table duplicate detection, and transaction-based validation involved issues the team had not previously encountered. To address them, the team gathered additional knowledge from SQL documentation, tutorials, and other technical resources, and applied that knowledge to the project. This process of learning and adaptation satisfies the EP4 criterion.

Table 4.3: Mapping with complex engineering activities.

#	CEA Attribute	Tick (✓)	Justification
1	**EA1: Range of resources** – Involve the use of diverse resources (people, money, equipment, materials, information, and technologies).	✓	The project required a range of resources, including team members, MySQL Workbench and phpMyAdmin (via XAMPP) for database design and management, MySQL as the database engine, Git for version control, a code editor (VS Code / PhpStorm) for PHP, HTML, CSS and JavaScript development, Mermaid for entity-relationship diagramming, PHPMailer for SMTP-based email verification, computers, SQL documentation, and online technical resources. These resources were used for database design, implementation, testing, debugging, and documentation throughout the project.
2	**EA2: Level of interaction** – Require resolution of significant problems arising from interactions between wide-ranging or conflicting technical, engineering, or other issues.	✓	The project required continuous interaction among team members through regular meetings, group discussions, and brainstorming sessions. When technical or design problems arose, the team met in person, studied together, and jointly analyzed the issues to discuss possible alternatives and develop suitable solutions. This collaborative interaction helped resolve significant project challenges and satisfied the EA2 criterion.

Chapter 5

Conclusion

This chapter concludes the development of ReliefX: Smart Flood Relief Management System by summarizing the major achievements of the project. It also discusses the current limitations of the system and identifies possible improvements for future development.

5.1 Summary

ReliefX: Smart Flood Relief Management System was developed to improve the management and coordination of flood-relief activities involving multiple NGOs. The system provides a centralized platform for managing beneficiaries, NGOs, relief items, stock, distribution points, and relief distribution records.
A major feature of the system is database-level duplicate prevention, which prevents a family from receiving the same category of relief within seven days using a stored procedure and triggers. The system also protects beneficiary identity information through salted cryptographic hashing, maintains an audit trail of blocked duplicate attempts, and provides an area-wise fairness view for analyzing relief distribution. Overall, the project demonstrates the practical application of DBMS concepts to improve data integrity, privacy, resource management, transparency, and fairness in flood-relief operations.

5.2 Limitation

Although the system provides the core functionality required for flood-relief management, it has some limitations. The current implementation is primarily designed as an academic prototype and has not been tested with a large number of real-world NGOs and beneficiaries. The system also depends on a centralized database and internet connectivity for multi-organization coordination. The current identity verification mechanism is based on hashed identity information and does not provide direct integration with any government identity-verification service. In addition, the fairness analysis is based on available population and distribution data and may not represent every real-world factor affecting relief requirements.

5.3 Future Work

The system can be further improved by introducing cloud-based deployment to support large-scale access from different locations. A mobile application could also be developed for NGO field workers to register beneficiaries and record distributions directly from relief sites. Future versions may include integration with government or authorized identity-verification services, real-time notifications, GPS-based distribution-point tracking, automated stock alerts, and more advanced analytics for identifying high-risk or underserved areas. The system could also incorporate machine learning-based analysis to predict relief requirements based on historical flood, population, and distribution data. These improvements would make ReliefX more scalable, intelligent, and suitable for real-world disaster-relief operations.

References

[1] Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.
Authentic Pearson Source

[2] Oracle Corporation. (2026). *MySQL 8.0 Reference Manual*. Oracle Corporation.
Official MySQL Reference Manual

[3] OWASP Foundation. (2024). *Cryptographic Storage Cheat Sheet*. OWASP Foundation.
Official OWASP Cryptographic Storage Guide

[4] International Federation of Red Cross and Red Crescent Societies. (2024). *Bangladesh: Flood Response and Recovery Operations*. IFRC.

[5] International Federation of Red Cross and Red Crescent Societies. (2024). *Step up in support needed as massive floods cause destruction in Bangladesh*. IFRC.

[6] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill.

[7] Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.

AI Tools Used
[8] OpenAI. (2026). *ChatGPT*. OpenAI.
Official ChatGPT Website

[9] Google. (2026). *Gemini*. Google.
Official Google Gemini Website

[10] Anthropic. (2026). *Claude*. Anthropic.
Official Claude Website